5. The Processor Interface

LOAD-STORE QUEUE

Sources:

- Smruti R. Sarangi. Next-Gen Computer Architecture (1st edition), White Falcon, 2023. ISBN 8119510143.

Memory Dependences

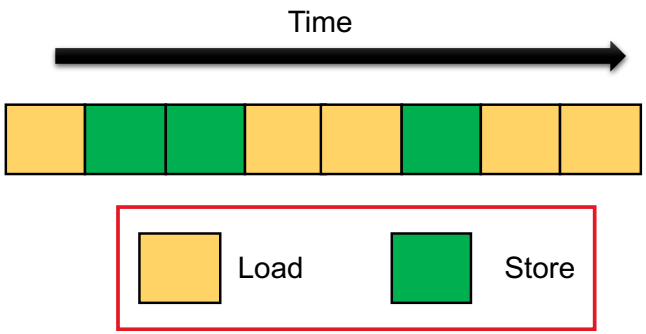
- Consider two memory instructions $I1$ and $I2$.
  - Generically, either **`ld <reg>, <addrmode>`** or **`st <reg>, <addrmode>`**.
- $I1 \xrightarrow{p} I2$.
- Under what conditions are these instructions dependent?
  - Dependence through registers.
  - Dependence through memory.

Various Cases

- Preconditions
  - $\mathrm{EA}(I1) = \mathrm{EA}(I2)$.
  - No $I3$ such that $I1 \xrightarrow{p} I3 \xrightarrow{p} I2$ and $\mathrm{EA}(I3) = \mathrm{EA}(I2)$.
  - Uniprocessor only.
- Cases based on opcodes of $I1$ and $I2$.
- When do we discover the in/dependence?
  - After EA computation, i.e., at the end of Execute.
- [RAR] $\mathrm{op}(I1) = \mathrm{LD}, \mathrm{op}(I2) = \mathrm{LD}$
  - Can reorder instructions.
  - Value from one load can be forwarded to the other load.
- [WAR] $\mathrm{op}(I1) = \mathrm{LD}, \mathrm{op}(I2) = \mathrm{ST}$
  - Cannot reorder instructions.
- [RAW] $\mathrm{op}(I1) = \mathrm{ST}, \mathrm{op}(I2) = \mathrm{LD}$
  - Cannot reorder instructions.
- [WAW] $\mathrm{op}(I1) = \mathrm{ST}, \mathrm{op}(I2) = \mathrm{ST}$
  - Cannot reorder instructions.
  - Maybe elide first store?

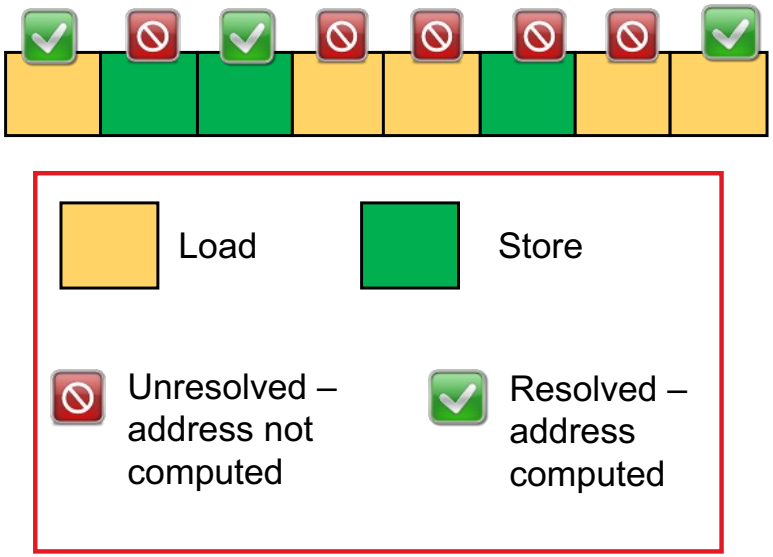
Load-Store Queue: Concepts

- LSQ sends loads and stores to the memory subsystem (L1 D$).
- Loads can execute immediately ... kinda.
- Stores must wait until we are sure that it is safe to update processor state.

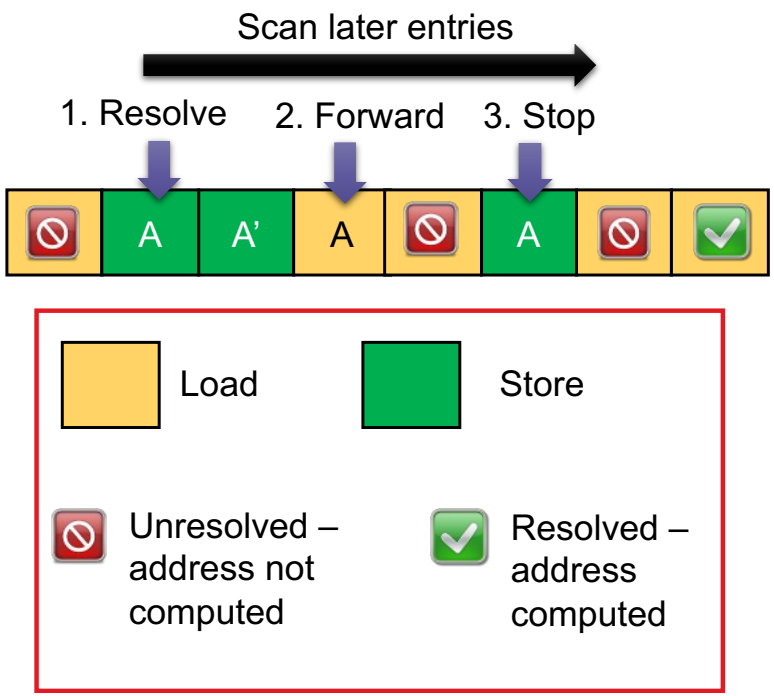


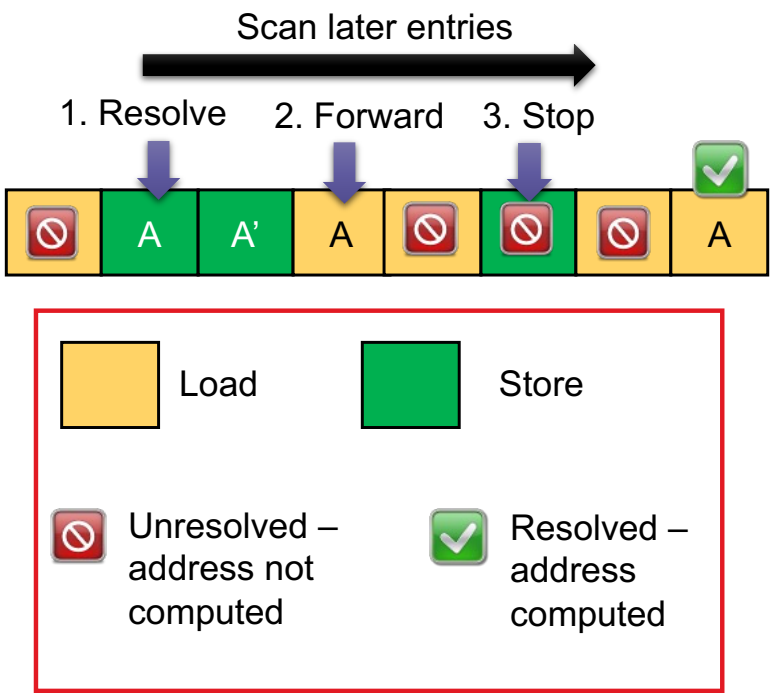
- Stores must wait.
  - So, they must be held in a storage area.
  - This area must be managed as a queue, so that stores maintain precise exceptions.
  - So, a load must first check this queue.
- Loads cannot be sent directly to the cache.
  - There might be earlier stores that are unresolved.
  - Such an unresolved store might resolve to the same EA.

Resolved and Unresolved Entries in LSQ

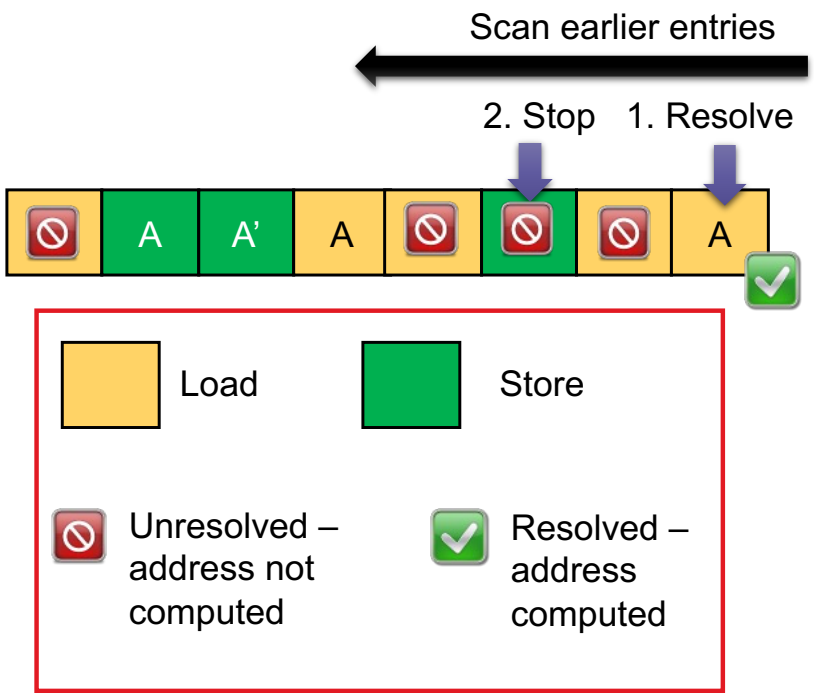


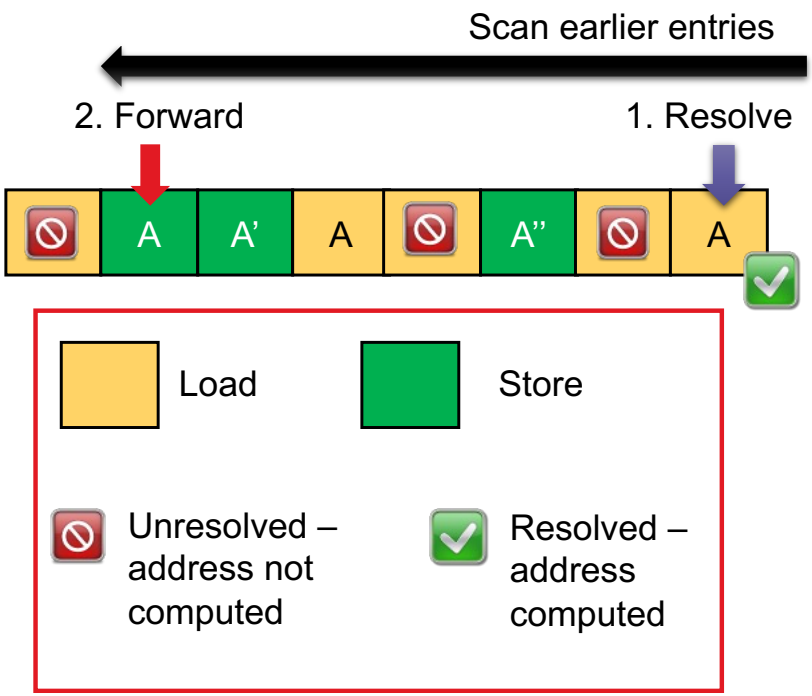
Resolution of Store (Two Cases)





Resolution of Load (Two Cases)



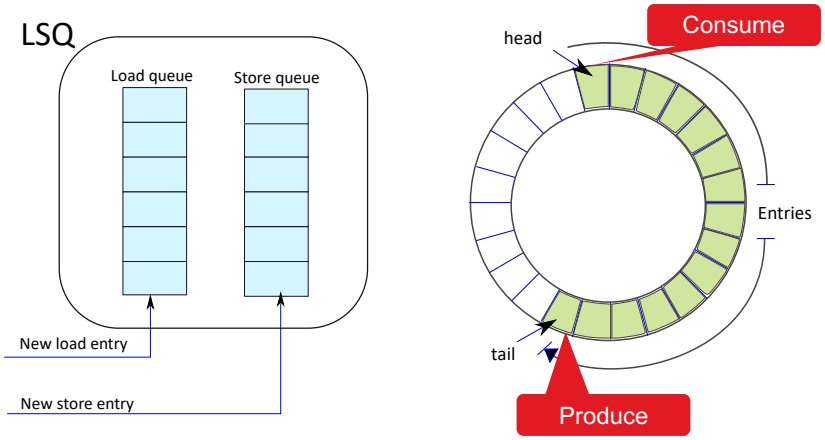


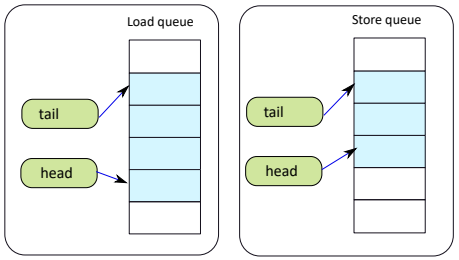
Summary of Actions

Type	Search Direction	Action: Condition
Loads	Search all the stores *before* it	1. Terminate and forward value: Store to the same address found. 2. Terminate: Store with an unresolved address found. 3. Go to D$: Conditions (1) and (2) not met.
Stores	Search all the loads and stores *after* it	1. Terminate: Store to the same address found. 2. Terminate: Store with an unresolved address found. 3. Forward value: Load from the same address found.

LSQ: Implementation

- For efficiency, use two queues: one load queue and one store queue.
- Queues are arranged as circular queues.
- Note how structure of entries maintains temporal order information.





Load queue entry

1. Load address
2. Index of the *tail* ptr in the store queue when the entry was added

Store queue entry

1. Memory address and value
2. Index of the *tail* ptr in the load queue when the entry was added

LSQ Mechanisms: Basic Operations

- Search for
  - all the stores before a load.
  - all the stores after a store.
  - all the loads after a store.
- Find operations that match a given EA.
- Find the first/last operation matching certain criteria.

- How do we compute such functions fast in hardware?
  - Think parallel!
  - Assume the availability of bit vectors *valid* and *resolved*.
  - Also, another bit vector *match* identifying all the entries that match a given EA.

Example Mechanisms: $before$ and $after$

- $before(j)$ returns a bit vector that contains a 1 for all valid queue entries that are before a given position j.
  - Uses helper function $prec(j)$, which returns a bit vector containing a 1 for all queue entries before position j.

$j \geq head$

$$before(j) = \overline{prec(head)} \wedge prec(j)$$

$j < head$

Wraparound

$$before(j) = \overline{prec(head)} \vee prec(j)$$

- $after(j)$ returns a bit vector that contains a 1 for all valid queue entries that are after a given position j.
  - Uses helper function $map(j)$, which returns a bit vector containing a 1 at position j and 0s elsewhere.

$j \leq tail$

$$after(j) = \overline{prec(j)} \wedge \overline{map(j)} \wedge (prec(tail) \vee map(tail))$$

$j > tail$

Wraparound

$$after(j) = (\overline{prec(j)} \vee prec(tail) \vee map(tail)) \wedge \overline{map(j)}$$

Implementing the LSQ

- Lots of operations can happen at each cycle.
  - Compute various functions and find a set of queue locations of interest.
  - Find either the first or last location.
  - Enqueue and dequeue memory operations.
  - Resolve memory operations on completion of EA calculation.
  - Handle limited queue size.
- How do we do all of this quickly in hardware?